



Wirtschaftswissenschaften
Fachgebiet Wirtschaftsinformatik & Operations Research

Multiagent Systems
Prof. Dr. Oliver Wendt
<http://bisor.wiwi.uni-kl.de/>

Taxation in Multi-Agent-Systems:
Comparison of an
independent planner
and
democratic agent voting
using the AI-Economist framework

Betreuer:

Konstantin Kloster, M. Sc.

Verfasser:

Lasse Cezanne, B. Sc.

cezanne@rhrk.uni-kl.de

Studiengang: Sozioinformatik M. Sc.

Eingereicht am:

01.10.2024

Table of Content

A. Introduction	1
B. Theoretical Background	2
B.1. Multi-Agent Systems	2
B.1.1. Cooperative Multi-Agent Systems	4
B.1.2. Competitive Multi-Agent Systems	5
B.2. Learning algorithms for MAS	6
B.2.1. Reinforcement Learning	6
B.2.1.1. Q-Learning	7
B.2.1.2. Deep Reinforcement Learning	8
B.2.1.3. Deep Q Networks (DQN)	8
B.2.1.4. Proximal-Policy Optimization (PPO)	8
B.3. AI-Economist Foundation Framework	9
C. Project setup	11
C.1. Scenarios	12
C.1.1. Tax policy using the Saez formula	12
C.1.2. Tax policy using a social planner	13
C.1.3. Tax policy by voting of egoistic agents	13
C.1.4. Tax policy by voting of (partly) utilitarian agents	13
C.1.5. Additional Free Market scenario	14
C.2. Training configuration	14
D. Results	15
D.1. Further validation testing	17
E. Discussion	19
F. Conclusion	20
Bibliography	21

A. Introduction

Taxation plays a critical role in shaping economic systems, influencing both individual behavior and societal welfare. In the context of multi-agent systems (MAS), where agents represent individuals or entities interacting with each other in a shared environment, designing optimal tax policies becomes an intriguing challenge. Multi-agent systems are widely used to simulate complex environments in which agents pursue their own goals, cooperate, or compete. In such systems, taxation mechanisms can help redistribute resources, promote equality, and optimize overall productivity. However, determining how to balance these objectives is a complex task, particularly when agents may have conflicting interests.

Multi-agent systems have found broad applications across various fields, ranging from economics and robotics to traffic management, distributed computing, and even environmental modeling. In economics, MAS are used to simulate complex market dynamics, allowing researchers and policymakers to study the effects of interventions such as taxation, subsidies, or regulations in controlled, scalable environments.

One of the key techniques that enables agents in MAS to learn, adapt, and make decisions autonomously is reinforcement learning (RL). RL is a machine learning paradigm where agents learn by interacting with their environment and receiving feedback in the form of rewards or penalties. Through trial and error, agents gradually learn policies that maximize their cumulative reward. In multi-agent systems, RL allows agents to develop sophisticated strategies as they respond not only to their environment but also to the actions of other agents. In competitive MAS, agents might learn adversarial strategies to outperform their opponents, while in cooperative MAS, they might learn to collaborate to achieve common goals.

In this project I utilize the proposed framework of (Zheng *et al.*, 2020) to investigate the differences between a taxation policy determined by a central planner with own objectives versus a direct democracy approach where agents determine a taxation policy directly via equal participation using Reinforcement Learning. In this context I also investigate the differences between the assumption of purely rational and egoistic agents and more utilitarian agents.

This work begins with an overview of the theoretical background, introducing multi-agent systems, their learning algorithms, and the technical MAS framework by Zheng *et al.* Following this, the project setup section outlines the design of the simulated scenarios as well as the setup and training configurations used. The results section presents the findings, followed by a discussion that contextualizes these results within the context of the theoretical background surrounding MAS. Finally I conclude by summarizing my work and offering perspectives about future work about this topic.

B. Theoretical Background

B.1. Multi-Agent Systems

The study of multi-agent systems (MAS) is concerned with the analysis of digital systems comprising multiple interacting intelligent agents within a clearly defined environment.

Formally we define a MAS as such:

Definition: *A multi-agent system (MAS) consists of a network of loosely-coupled computational autonomous agents who can perform action, they have resources at their disposal and they possess knowledge, capabilities or skills. They are situated in a common environment and they can interact through a set of rules, namely an interaction protocol.* (Fasli, 2007, p. 76)

Intelligent agents ordinarily need to fulfill the following properties:

- Agents having control over their own actions and internal states (**Autonomy**)
- Agents being able to adapt to their environment in order to follow their objectives (**Reactivity**)
- Agents engaging in goal-directed behavior by demonstrating initiative in pursuing their design objectives (**Proactiveness**)
- Agents being capable of interaction with other agents to satisfy their design objectives (**Social ability**)

(Fasli, 2007; Wooldridge, 2009)

In this project paper we only consider episodic environments, in which agents observe the environmental states and take actions in discrete time steps. In this context the behavior of agents can be formalized by four different functions:

1. see: $S \rightarrow P$
2. next: $IS \times P \rightarrow IS$
3. action: $IS \rightarrow A$
4. do: $A \times S \rightarrow S$

$S = \{s_1, s_2, \dots\}$ describes the set of possible environment states and P the set of corresponding perceptions for an individual agent. IS describes the set of internal states of an agent that are updated by their perceptions. Meanwhile $A = \{a_1, a_2, \dots\}$ is the set of possible actions of an agent.

In this sense intelligent agents generate observations of the environment (see: $S \rightarrow P$), that influence their internal state (next: $IS \times P \rightarrow IS$) and therefore affect their chosen actions (action: $IS \rightarrow A$). This chosen action then manipulates the environment to create a new state (do: $A \times S \rightarrow S$).

In a **multi**-agent environment, there is also the question of the order of these steps for the different agents. Do the agents all act synchronously or one after the other? Oftentimes for simplicities sake or because this allows some parallelization during training the agents act simultaneously, first all generating observations and then choosing an

action at the same time, although this might not always be possible if certain actions of agents depend on other agents acting beforehand (like a trader requiring offers and demands to act).

One can also differentiate between MAS employing homogenous or heterogenous agents. In a homogenous setting all agents are constructed similar and possess the same action-set, while in a heterogenous setting the agents possess different possible actions and behave differently from each other. In a hybrid setting there might be multiple groups of homogenous agents. It is important to note that even in a homogenous setting while the agents possess the same actions and might start with similar parameters during training they can still specialize and utilize different behavior.

In general, MAS can be characterized by a number of different aspects (different taxonomies), which are also variously identified in the literature, such as agent or system characteristics (for further information see (Panait and Luke, 2005)).

An important component of MAS is the interaction and communication between agents. Theoretically communication in a MAS might be unrestricted, but if every agent would know all possible information about every agent, this might be isomorphic to a single-agent system. Therefore "a true multi-agent problem necessitates restrictions on communication" (Panait and Luke, 2005). Communication both increases the perceptual space of an agent (by adding information from other agents) and increases the action space (by adding actions of communication to other agents). One can differentiate between *direct communications* and *indirect communications*.

Direct communications comprises communication methods by which agents can directly share information with one another like a shared blackboard or message-passing. Depending on the setting, agents might need to be incentivized to actually use these communication methods for success.

Indirect communications meanwhile include communication methods which "involve the *implicit* transfer of information from agent to agent through modification of the world environment." With this definition in mind a dedicated shared channel might not be necessary to facilitate communication (Panait and Luke, 2005).

Another important aspect in the context of MAS is the concept of **utility**. The current utility of an agent describes their valuation of the perceived environment state. As such we can define a utility function for each agent as follows:

$$\text{utility}(u): P \rightarrow \mathbb{R}$$

In literature the utility function is also often depicted as $u(s)$, so a valuation of the actual environment state instead of the agents perception. The exact scaling of the utility function is meanwhile not relevant, only the preferences between different possible environment states need to be clearly defined. Therefore for two different states s_1 and s_2 , an agent is indifferent between those states if $u(s_1) = u(s_2)$ and prefers the state with the bigger utility over the other if they are not equal (Fasli, 2007).

B.1.1. Cooperative Multi-Agent Systems

In cooperative MAS different agents work together to fulfill a common goal, solve a joint task or maximize their common utility. An example would be a team of agents trying to forage resources in a restricted environment (a so-called foraging task). One option for constructing a cooperative environment is to define a common and shared utility for all agents that needs to be maximized in order to facilitate cooperation (for foraging this might indicate the value of all collected resources). There however exist different methods to train Cooperative Multi-Agent Systems. Following the taxonomy of (Panait and Luke, 2005) one can differentiate between *team learning*, where a single learner searches for behaviors of the entire team of agents and *concurrent learning*, where multiple concurrent learning processes are employed.

Since in team learning there is only one learner, traditional machine learning techniques can be applied for training, however the search space during the learning process might be quite big, limiting the learning performance. Additionally the centralized nature of the learner requires that all resources need to be available at a single place for computation (making parallelization difficult).

One can further differentiate team learning into *homogeneous*, *heterogeneous* and *hybrid team learning*. In homogeneous team learning only a single agent behavior is developed that is applied to all agents while in heterogeneous team learning the learner develops a unique action strategy for each agent. Since there exists a tradeoff between the simplicity of the homogeneous and the complexity of the heterogeneous approach, hybrid solutions exist where the agents are separated into different teams where all agents in a team get assigned the same behavior, limiting the search space while allowing for specialization of agent behavior (Panait and Luke, 2005).

In concurrent learning meanwhile it is typical that each agent possesses their own unique learning process to find their optimal behavior. A difficult problem to overcome in concurrent learning is that "as the agents learn, they modify their behaviors, which in turn can ruin other agents' learned behaviors by making obsolete the assumptions on which they are based" (Panait and Luke, 2005) While agents can view other agents as part of a dynamic environment to which they must adapt, the adaptations of agents to this very environment may again change the environment itself which might make those adaptations invalid. (Panait and Luke, 2005) propose three main thrusts of research in the area of concurrent learning: the *credit assignment problem*, challenges in the *dynamics of learning* and the *modeling of other agents*.

The credit assignment problem deals with the question of how to distribute the reward obtained at a team level to individual learners. In the global reward approach the team reward is equally split among all learners (also called fully-cooperative scenario). Another approach is called local reward in which agents only receive a reward for tasks they have accomplished by themselves. Depending on the situation and the chosen goal for the MAS a global and local reward can be combined for maximum efficiency. The dynamics of learning in a MAS with a concurrent learning approach depend on the chosen credit assignment paradigm (so global reward, local reward or a combination

of both). In the fully-cooperative scenario with a global reward we can assume that the agents are able to reach an optimal equilibrium with the right learning approach that maximizes their shared utility/reward. In so-called general sum games on the other hand, where the increase of an agents reward might be connected to the decrease of another agents reward even in a cooperative environment, it can however not be easily shown that a chosen learning algorithm will lead to an optimal solution but rather only local sub-optimal Nash equilibria might be achieved.

The question of how detailed an agent models other agents in the environment is important to raise because the most detailed description of another agent would include the other agent modelling the original agent which again models the other agent, which leads to an infinite recursion. It is concluded that it is best when modelling other agents that learning agents should minimize the assumptions about other agents' policies. If there is a large number of agents, it might also be simplest to model other agents as identical to the modeling agent (Panait and Luke, 2005).

B.1.2. Competitive Multi-Agent Systems

In competitive MAS agents compete with each other and do not share a common goal, but rather individualized goals. This might sound similar to the setting of a cooperative MAS with completely local rewards with the difference that in a competitive MAS the intention of the system designer is for the agents to not cooperate but rather compete with each other. In that sense the intention of the designer of a MAS might characterize it to be either cooperative or competitive, depending on whether the learning system is constructed with the intent to encourage cooperation among the agents or not (Panait and Luke, 2005).

The intent of a competitive MAS is often rather to pit multiple agents against on another to "hopefully [establish] a beneficial 'arms race' among them" (Panait and Luke, 2005). An example would be agents playing board games like chess against each other and meanwhile improving upon each others performance. This is also called competitive coevolution.

An important aspect of learning in competitive MAS is something called *loss of gradient* where one learner dominates the other learner such that the dominating learner always wins and the dominated learner always loses which leads to neither of the agents learning anything.

Progress in a competitive coevolution is also difficult to measure and monitor since an agent winning more often does not necessarily mean that they found a good strategy, rather the competing agent might be only employing rather bad strategies (Panait and Luke, 2005).

Finally, due to non-transitive relationships between agent interactions, competition can lead to cyclic behavior, where agents circle about one another. "For example, in the rock-scissors-paper game, Agent A picks Rock and Agent B picks Paper. Agent A then adapts to Scissors, causing Agent B to adapt to Rock. Then Agent A adapts to Paper, Agent B adapts to Scissors, and Agent A adapts again to Rock." (Panait and Luke, 2005)

B.2. Learning algorithms for MAS

We have already covered learning approaches for MAS like team learning or concurrent learning, however this needs to be differentiated with the choice of an appropriate learning **algorithm** for training a MAS. These learning algorithms can be categorized into different categories of algorithms that are commonly used to train MAS.

Most learning algorithms for MAS utilize machine learning. According to (Panait and Luke, 2005) there are three main approaches to machine learning for MAS: *supervised learning*, *unsupervised learning* and *reward-based learning*. "These methods are distinguished by what kind of feedback the critic provides to the learner. In supervised learning, the critic provides the correct output. In unsupervised learning, no feedback is provided at all. In reward-based learning, the critic provides a quality assessment (the 'reward') of the learner's output." (Panait and Luke, 2005) The majority of MAS papers employ reward-based learning algorithms that either utilize reinforcement learning or stochastic search methods such as evolutionary algorithms.

B.2.1. Reinforcement Learning

Reinforcement learning algorithms, because of their general applicability and robust nature, are among the most commonly used algorithms for training MAS (Claus and Boutilier, 1998).

As a *reward-based learning* approach reinforcement learning (RL) utilizes a *reward* for performing particular actions to train agents. RL methods use expected utilities to explore the state space of an environment. They continuously update these expected utilities during training. Often, the update will be a weighted sum of the current utility, the reinforcement received by performing the action, and the expected utility of the next state reached after performing the action (Panait and Luke, 2005).

The general process of Multi-Agent Reinforcement can be visualized by Figure 1, where actions by agents influence the environment to create new states which results in different rewards and updated states for the agents from which to reinforce their future behavior. In this sense the concept of a reward is closely linked to that of utility as discussed in Section B.1.

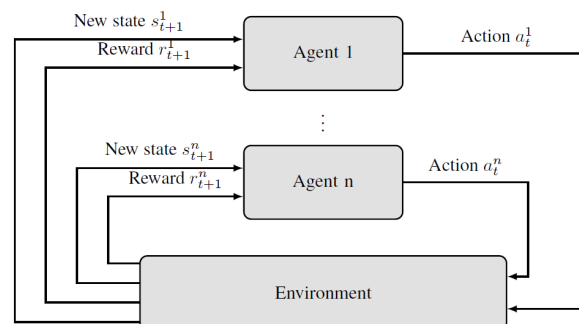


Figure 1: Multi-Agent Reinforcement Learning¹

¹This figure was taken from the project topic overview sheet provided by Konstantin Kloster, Chair of Business Information Systems & Operations Research RPTU

Multi-agent reinforcement can be modeled via a Markov decision process expressed in the tuple $\langle S, A, P, R \rangle$:

1. S represents the *state space* of all possible states of the environment including the states of the individual agents (see Section B.1)
2. $A = A_1 \times A_2 \times \dots \times A_n$ represents the joined *action space* of all agents where A_1 represents the actions space of agent₁
3. $P(s' \mid s, a_1, a_2, \dots, a_n)$ is the probabilistic *transition function* that defines that probability of changing from environment state s to state s' given the agents joint action $a = (a_1, a_2, \dots, a_n)$. This can also be a deterministic function if the environment is defined without probabilistic chance involved.
4. $R(s, a_1, a_2, \dots, a_n)$ represents the *reward function* that maps the current environment state and the chosen joint action a of all agents to an immediate reward for each agent (similar to the utility function in Section B.1).

Additionally to not only consider immediate rewards but also future rewards, because we want to utilize the expected utility of the agents behavior, a *discount factor* $\gamma \in [0, 1]$ is introduced that determines how future rewards are weighted compared to immediate rewards, where a higher γ means future rewards are considered as more important.

In reinforcement learning the *policy* of each agent is optimized in order to maximize their expected utility or reward. An agents policy $\pi_{i(a_i \mid s)}$ is defined as a mapping from the state space S to a probability distribution over actions A_i . The joint policy $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ is the collection of all agents policies.

In a purely cooperative setting (see Section B.1.1) where all agents share the same reward function the objective of the learning algorithm is to find the optimal joint policy that maximizes the expected utility of all agents which can be formulated as:

$$\max_{\pi} \mathbb{E} \left[\sum_{[t=0]}^{\infty} \gamma^t R(s_t, a_1^t, a_2^t, \dots, a_n^t) \right]$$

In a competitive setting (see Section B.1.2) meanwhile it is the objective of the learning algorithm to find an optimal policy for each agent that optimizes their respective expected utility which can be formulated as:

$$\max_{\pi_i} \mathbb{E} \left[\sum_{[t=0]}^{\infty} \gamma^t R_i(s_t, a_1^t, a_2^t, \dots, a_n^t) \right]$$

There exists a magnitude of different algorithms for reinforcement learning in MAS like Q-learning, Deep Q Networks (DQN), Value-Decomposition Networks (VDN) or Proximal-Policy Optimization (PPO).

B.2.1.1. Q-Learning

Q-Learning is one of the most fundamental RL algorithm. It is a tabular model-free RL algorithm where an agent learns a so-called **Q-function** that represents the expected cumulative reward for taking a certain action in a given state. The goal of the algorithm is to find the optimal policy by iteratively updating the so-called **Q-values** in the **Q-**

table. The Q-table is the tabular representation of the Q-function where each entry corresponds to a state-action pair. For a given state s , the agent chooses the action a that maximizes the Q-value, i.e., $Q(s, a)$. Q-values are updated using the so-called Bellmann equation which considers current and possible future rewards for a state-action-pair with the use of a *learning rate* and *discount factor* parameters. Q-Learning typically uses a so-called ϵ -greedy strategy, where the agent explores randomly with probability ϵ and exploits its knowledge by selecting the action with the highest Q-value otherwise. For more details on Q-learning in distributed systems see (Lauer and Riedmiller, 2000).

B.2.1.2. Deep Reinforcement Learning

Deep Reinforcement Learning (DRL) refers to the integration of deep learning with reinforcement learning techniques to enable agents in multi-agent environments to learn more complex behaviors and decision-making strategies. Deep learning, particularly deep neural networks, is/are used to approximate the complex functions that map states (or state-action pairs) to values or actions. In DRL, deep neural networks (such as convolutional or recurrent neural networks) are used to approximate the policy or value functions that guide agent decisions. These deep neural networks can be leveraged to handle high-dimensional inputs like images or complex state spaces where tradition RL techniques like Q-learning might struggle.

B.2.1.3. Deep Q Networks (DQN)

DQNs represent an extension of Q-learning to the realm of DRL. They extend Q-Learning by using deep neural networks to approximate the Q-function, allowing them to handle high-dimensional and continuous state spaces. Instead of storing the Q-values in a table, DQN uses a deep neural network to approximate the Q-function. The input to the network is the state s , and the output is the Q-values for each possible action a in that state. This allows DQN to generalize across large or continuous state spaces, such as image-based inputs. Like Q-Learning, DQN also employs an ϵ -greedy strategy, but it typically anneals (decays) ϵ over time to balance exploration and exploitation more effectively. Because of its increased complexity, training might suffer from instability which is counteracted by techniques such as *experience replay* and of separate *target networks* to stabilize training and improve sample efficiency (Mnih et al., 2013).

B.2.1.4. Proximal-Policy Optimization (PPO)

The Proximal Policy Optimization (PPO) algorithm (Schulman et al., 2017) is a widely-used reinforcement learning (RL) method that belongs to the policy gradient family. While DQN is a value-based method that learns an approximation of the Q-value function, PPO is a **policy-based** method that directly learns a policy, i.e. a probability distribution over actions given a state, and tries to find the optimal policy that maximizes the expected reward without explicitly estimating Q-values. This allows training of MAS both for discrete and continuous action spaces.

PPO uses a policy network to directly model the agent's policy, which outputs the probability distribution over actions. Actions are sampled from the learned policy, which can

be stochastic. This allows more exploration by inherently maintaining some randomness in action selection. The Centralized Training with Decentralized Execution (CTDE) approach allows agents to learn using global information during training, but they execute their policies using only local observations once training is complete. During training, a centralized critic can be used to evaluate the policies of all agents simultaneously. The critic has access to the global state of the environment, including the actions and observations of all agents, allowing it to compute a more accurate advantage function for each agent. This helps each agent update its policy while considering the influence of other agents. Meanwhile each agent's policy (or actor) is decentralized, meaning that at execution time, each agent makes decisions based solely on its own local observations and not the global state or the actions of other agents. This makes the approach scalable to large MAS environments. PPO additionally uses a clipped objective function to prevent large, destabilizing policy updates.

Compared to DQNs the PPO-algorithm is more flexible and robust, handling continuous as well as discrete action spaces and ensuring stable policy updates in a wider range of environments.

B.3. AI-Economist Foundation Framework

Foundation² is an economic simulation framework for MAS developed by Stephan Zheng et al. and used as part of the AI Economist in different scientific papers like (Zheng et al., 2020). While the project has recently been abandoned by its developers (last commit on GitHub happened in May 2022 and the repository has been archived in November 2023 - the project page is also no longer available), its core features do still work and the framework can be used to easily model flexible, modular and composable environments modelling socio-economic behaviors and dynamics.

The framework offers base classes to define different agents, components and scenarios for creating environments for socio-economic analysis with MAS.

It offers two implementations for *agents* representing economic actors in a defined environment. First it offers a mobile agent (representing a worker-type agent) of which multiple instances can be instantiated in an environment and secondly a so-called social planner (representing a government-type agent) of which only one instance can be instantiated per environment. Further types of agents can be implemented using the framework and provided base classes.

Further elements of the framework include *entities*, *components* and *scenarios* that can be used to build individualized environments or use pre-existing defined scenarios.

Entities represent endogenous and exogenous components of the environment such as labor, landmarks or resources. *Components* are used to add dynamics to the environment and define how agents can interact with environment via components. *Scenarios* compose Components to define specific dynamics of an environment and can be used to compute rewards and enable visualization of experiment results.

²<https://github.com/salesforce/ai-economist> (last accessed 23th of September 2024)

The framework also includes an environment wrapper that can be used to convert the defined and constructed scenarios and environment inside of the framework to a format that can be used by RLLib³ library to facilitate the usage of RL algorithms from their repertoire.

The MAS constructed using this framework are neither clearly competitive nor cooperative due to the framework's socio-economic nature that involves both cooperative and competitive behavior. Since different environments can be implemented using this framework the classification of the resulting MAS is dependent on the intent of the developer of these systems. The base scenarios feature a competitive set of mobile agents trying to maximize their individual rewards and a social planner trying to maximize social metrics that could be interpreted as a cooperative or competitive reward scheme.

³<https://docs.ray.io/en/latest/rllib/> (last accessed 23th of September 2024)

C. Project setup

This project was inspired and is heavily influenced by (Zheng *et al.*, 2022). In (Zheng *et al.*, 2022) the authors proposed a two-level deep multi-agent reinforcement learning design to find optimal taxation policies with the help of their AI Economist and the Foundation framework (see Section B.3).

In this project I want to utilize their framework to investigate the differences between a taxation policy determined by a central planner with own objectives versus a direct democracy approach where agents determine the taxation policy directly via equal participation. In this context I also investigate the differences between the assumption of purely rational and egoistic agents and more utilitarian agents.

This experiment uses some of the provided base components from the Foundation framework while also developing new and extended versions. I chose to use the **one-step-economy** environment from Foundation because of its simplicity and focus on the sole purpose of taxation policy and labor investment. The environment is also shortly described and used in (Zheng *et al.*, 2022) to test and validate their proposed learning design. Performing training using the one-step-economy is also less demanding on the utilized hardware.

I created four different scenarios and trained agents inside of these to compare the resulting MAS and how the tax policies in these developed:

1. Tax policy using the Saez formula
2. Tax policy using a social planner
3. Tax policy by voting of egoistic agents
4. Tax policy by voting of (partly) utilitarian agents

The first and second scenario were also used for validation in (Zheng *et al.*, 2022). Due to limited hardware capabilities I reduced the amount of agents and training iterations in both scenarios and repeated training to get comparable data to the latter two scenarios. While (Zheng *et al.*, 2022) created 100 agents and trained these for 60,000,000 iterations I reduced this to 9 agents and 1,000,000 iterations respectively.

In general the one-step-economy environment functions very similar in all four scenarios. The base scenario utilizes the two base components of Foundation *PeriodicBracketTax* and *SimpleLabor*. All scenarios consist of two steps:

1. Taxes are set via *PeriodicBracketTax*
2. Agents select how much to work/earn via *SimpleLabor*

Taxes are collected via a fixed income tax bracket inspired by the 2018 US Federal taxes. For each bracket a marginal tax rate can be set from a discrete set of options (0%, 5%, 10%, ..., 100%). Taxation is applied equally to all agents and the raised taxes are evenly distributed back to all agents.

With the *SimpleLabor* component agents can decide to invest labor to gain income. Each individual agent has an additional attribute *skill* that describes how proficient they are at working. Each Agent earns income proportional to the product of its labor

amount (representing hours worked) and its skill (representing wage), with higher skill and higher labor yielding higher income. Skill is distributed over all agents using samples from a Pareto distribution. Agents can choose from a discrete set of labor actions choosing to work between 1 and 100 “hours”. Since investing labor to gain income also comes with a cost there exists an optimal amount of labor invested for work for each individual agent depending on the current taxation schedule and an agents skill. During training agents try to find the optimal amount of labor invested.

C.1. Scenarios

C.1.1. Tax policy using the Saez formula

The Saez formula (Saez, 2001) uses an analytical formula that computes optimal tax rates for a given tax bracket in an idealized one-step-economy given a redistributive preference for weighting of individual utilities. The exact functionality and implementation of empirical and estimated parameters as well as the formula itself can be reviewed in (Zheng *et al.*, 2022, p. 16). For the purpose of this project it is only important to consider that the Saez formula can estimate theoretically optimal marginal tax rates which serves as a baseline to compare to the further scenarios.

This scenario follows the following two steps:

1. The marginal tax rates for each tax bracket are set via the Saez formula using empirical income distribution and elasticity of income with respect to taxation
2. Agents select how much to work/earn depending on the chosen tax rates and their individual skill to maximize their utility

Each agent only has one decision to make, namely how much labor to invest for income. They optimize this decision with a reward function that increases linearly with income and decreases as a power of labor (see Figure 2 - optimal labor invested is marked with a star). Agents observe their own skill level and current marginal tax rates as well as the income of all agents (pre-taxes) but lack further information about other agents.

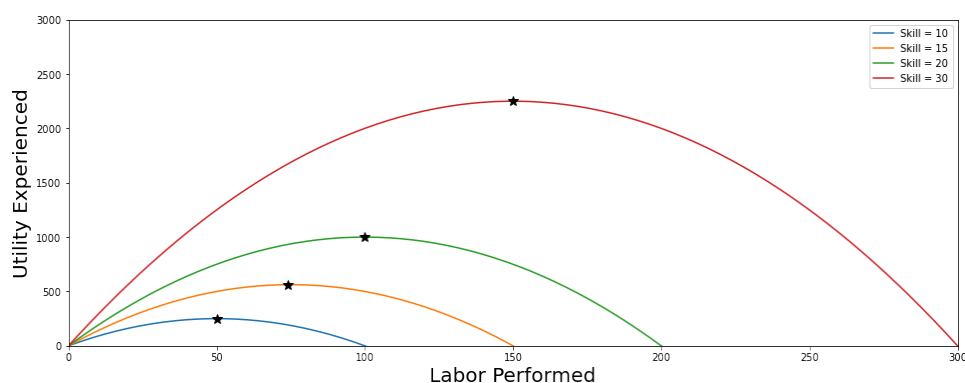


Figure 2: Mapping utility to labor performed for different skill-levels⁴

⁴This figure was generated using a modified notebook provided at https://github.com/salesforce/ai-economist/blob/master/tutorials/optimal_taxation_theory_and_simulation.ipynb

C.1.2. Tax policy using a social planner

This scenario is the base scenario used in (Zheng *et al.*, 2022) to test the robustness of the AI-Economist against the usage of the Saez formula.

This scenario consists of the following two steps:

1. The marginal tax rates for each tax bracket are set by a social planner agent that optimizes the chosen rates to maximize social welfare
2. Agents select how much to work/earn depending on the chosen tax rates and their individual skill to maximize their utility

To maximize social welfare the social planner uses the inverted income weighted utility which takes the average of all agents utility (defined as agent reward in Section C.1.1) weighted by the inverse of an agents income (pre-tax). The social planner can make its decision about the marginal tax rates based on observations about social metrics such as productivity and equality (measured with the Gini index) as well as all information observed by each agent. The behavior of the other agents is identical to the one described in Section C.1.1.

C.1.3. Tax policy by voting of egoistic agents

Instead of utilizing the two-level design in (Zheng *et al.*, 2022), in this scenario all decisions are made by agents and without a centralized social planner.

This scenario consists of the following two steps:

1. The marginal tax rates for each tax bracket are set as the average of all agents choices for tax rates for each respective bracket - agents optimize their individual choices to maximize their utility
2. Agents select how much to work/earn depending on the chosen tax rates and their individual skill to maximize their utility

While rewards and observations for agents remain the same, they now have the additional action space of choosing a marginal tax rate for each tax bracket similar to the social planner (discrete choice in 5%-steps). The actual marginal tax rates are then computed by averaging the individual agents choices for each bracket. For the second step the agents behavior is again identical to the one described in Section C.1.1.

C.1.4. Tax policy by voting of (partly) utilitarian agents

Similarly to the previous scenario all decisions are made by the agents without the involvement of a centralized social planner. However this scenario modifies the observations and rewards of agents to incorporate the consideration of social welfare.

This scenario consists of the following two steps:

1. The marginal tax rates for each tax bracket are set as the average of all agents choices for tax rates for each respective bracket - agents optimize their individual choices to maximize some composite of their utility and social welfare
2. Agents select how much to work/earn depending on the chosen tax rates and their individual skill to maximize some composite of their utility and social welfare

For this scenario the reward function was modified to both incorporate individual utility as described Section C.1.1 as well as social welfare as described in Section C.1.2. The reward is defined as a weighted sum of utility and social welfare (defined as inverse income weighted utility) using the additional parameter `utilitarian_weight`. This results in:

$$\text{reward} = \text{utilitarian_weight} * \text{social welfare} + (1 - \text{utilitarian_weight}) * \text{utility}$$

Since the chosen social welfare indicator is defined as the inverse income weighted utility both metrics should scale approximately similar. For this project in all further experimentation the `utilitarian_weight`-parameter is set to 0.5.

The agents observations are updated to account for necessary information that the social planner also received to choose appropriate marginal tax rates and evaluate their choice. In this scenario agents additionally observe social metrics such as productivity and equality (measured with the Gini index) but gain no further information about other agents. Even though this decision limits the agents knowledge slightly compared to the social planner, I chose this limitation to reflect real-life restrictions of this type of information for individuals and to reduce the amount of parameters for learning in the hopes to stabilize the training process.

C.1.5. Additional Free Market scenario

Additionally I created another scenario without an existing tax policy, where no taxes and redistribution exist. This scenario is called “Free Market” scenario in (Zheng *et al.*, 2022). This scenario is used to compare productivity metrics of agents to the other scenarios to get another baseline of performance.

C.2. Training configuration

The AI-Economist Foundation framework provides a training script and I used modified config-files from (Zheng *et al.*, 2022) for the training of scenarios. As already mentioned I reduced the amount of simulated agents from 100 to 9 and the amount of training iterations from 60,000,000 to 1,000,000 due to hardware restrictions. To enable exploration during early training an entropy coefficient schedule is used, which I also adapted to the reduced amount of training iterations. More details about the chosen hyper-parameters which I adapted from (Zheng *et al.*, 2022) can be reviewed in the attached GitHub-project⁵.

The hardware used for training is a Acer Swift 3 Laptop with an AMD Ryzen 5 4500U processor with 6 cores and 16 GB of memory.

I chose PPO as training algorithm mainly because of its utilization in the original paper (Zheng *et al.*, 2022), so that I could more easily compare my results with existing results and to have appropriate hyperparameters readily available. Additionally as explained in Section B.2.1.4 PPO offers a flexible and robust training approach for wide area of use cases and environments.

⁵<https://github.com/La-Cezanne/ai-economist-democratic-voting> (last accessed 01.10.2024)

D. Results

The development of important metrics during training can be reviewed in Figure 3.

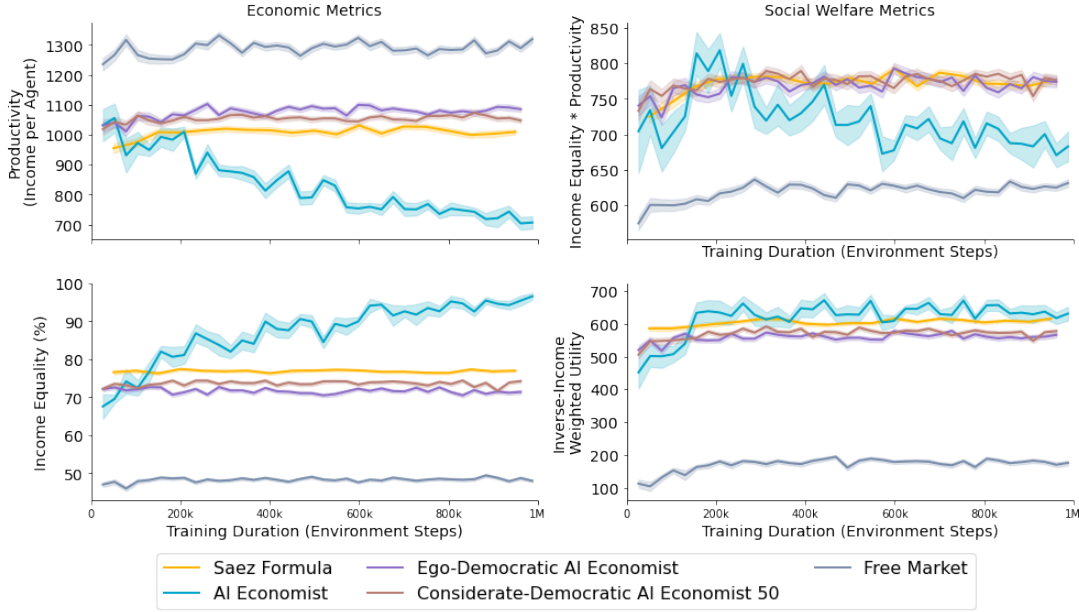


Figure 3: Important metrics during training⁶

As can be seen in Figure 3, all scenarios except for the tax policy using a centralized social planner (AI Economist in blue) reach a plateau rather quickly, while oscillating slightly. Interestingly the metrics for both scenarios with the agents voting for a tax policy (Ego-Democratic AI Economist and Considerate-Democratic AI Economist) show fairly similar results with only minor differences. The scenario with the tax schedule set by the Saez Formula shows slightly lower productivity with slightly improved social welfare metrics when considering inverted income weighted utility and income equality. The most dynamic behavior can be seen when considering the original scenario with a centralized social planner. While overall productivity decreases during training, the income equality increases during the whole training process. When considering social welfare metrics, the inverted income weighted utility increases for approximately 400,000 iterations and reaches a plateau above the other scenarios, while the product of income equality and productivity follows a similar path to other scenarios until about 400,000 iterations where it continually decreases. As expected with the Free Market scenario without taxes productivity is highest because investing additional labor is not disincentivized due to taxes being present. Meanwhile all social welfare metrics are lower without taxes as income is not redistributed to less skilled agents.

These results indicate little difference between the behavior of agents in the democratic voting scenarios even though their reward functions differ substantially (see Section C.1). While the scenario with the utilization of the Saez Formula seems to converge rather quickly, the original AI Economist scenario might still need further training to converge to a stationary behavior.

⁶Figure inspired by Fig. 2 in (Zheng et al., 2022)

The resulting tax brackets after training (see Figure 4) also show little difference between both democratic voting scenarios. Both scenarios result in a tax schedule with approximately 50% taxes in all brackets. The tax schedules for the first two scenarios (Saez Formula in yellow and AI Economist in blue) have more resemblance to the resulting tax brackets from (Zheng *et al.*, 2022). While (Zheng *et al.*, 2022) found that after their training both the scenario with the Saez Formula and the AI Economist resulted in the same tax schedule for the one-step-economy, I could not reproduce this result. This might either result from not enough training iterations or my choice of different (hyper-)parameters like the different amount of agents used.

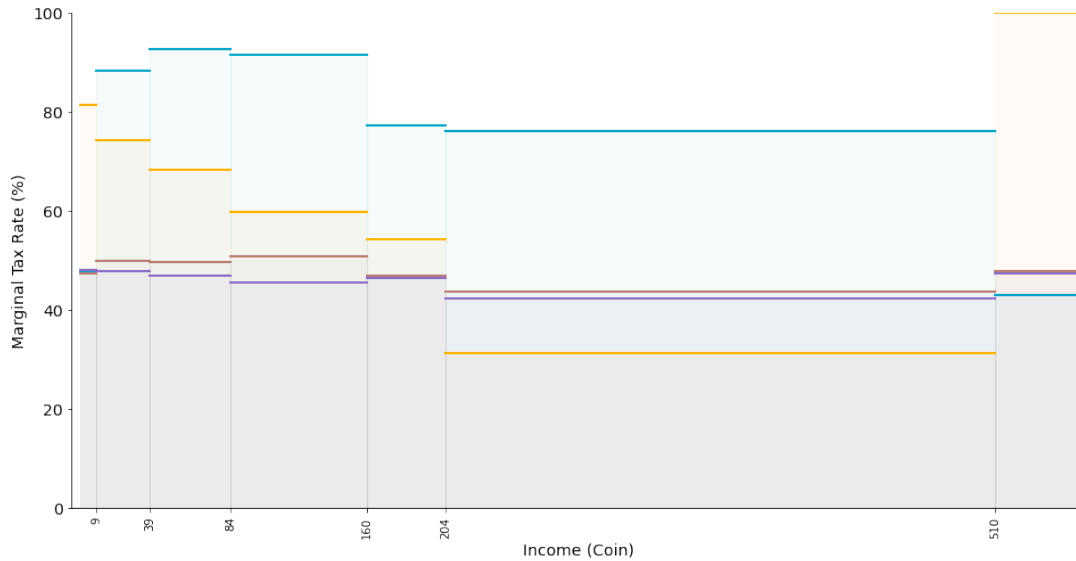


Figure 4: Resulting average tax bracket after training⁷

When considering the latter two scenarios with democratic voting, there exist multiple possibilities why a tax schedule of on average approximately 50% resulted from training:

1. Agents might pick marginal rates for a tax bracket at random resulting in an average of 50%
2. Half the agents mostly pick marginal rates above 50% and half below 50% with the same deviation from 50%
3. All agents pick marginal rates approximately at 50%

The first case would indicate that no real learning regarding marginal rates happened during training since agents just pick random marginal rates, while the second and third case would indicate some case of learning where the resulting 50% would be the result of the averaging of agent votes.

To investigate the question which of these cases is correct, I sampled 1000 instances of voting marginal tax rates for each bracket and each agent and calculated the mean and standard deviation of these samples.

⁷Figure inspired by Fig. 3B in (Zheng *et al.*, 2022)

The results (see Figure 5) show a close resemblance to a purely random sampling indicating that no real learning regarding marginal rates happened during training for both democratic voting scenarios. I would have expected agents with less skill (meaning less income from labor) to have learned to vote for higher marginal tax rates in higher tax brackets so as to get more utility from redistribution in the egoistic voting scenario and even higher marginal tax rates in higher brackets in the scenario with (partly) utilitarian agents due to an increase of income redistribution and social welfare in such a case.

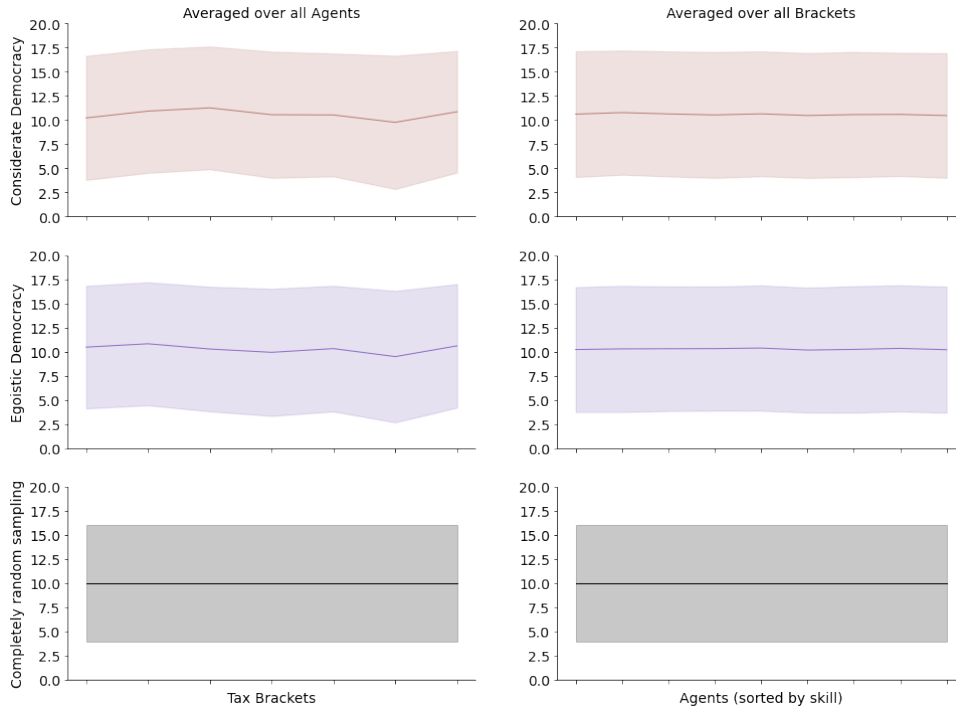


Figure 5: Tax rate voting by agents in different scenarios after training

To further test whether these results were produced by simply not enough training or rather a faulty design with democratic voting agents leading to such a complex environment that agents are unable to converge to a stationary result, I created another test scenario:

D.1. Further validation testing

I created another one-step-economy scenario with democratic voting by egoistic agents (see Section C.1.3) but reduced the amount of available tax brackets to two (income ≤ 1000 and income > 1000) and the amount of possible voting actions per bracket to three (0%, 50% and 100%). The averaging of marginal tax votes per bracket was set up such that marginal tax rates would be clipped to values between 10% and 90%. I also reduced the amount of possible labor options to further reduce the action space and manipulated the skill distribution of agents in such a way that only a minority of agents would reach the higher tax bracket. Then I increased the amount of training iterations to 2,000,000.

I expected that with this decreased action space and additional learning iterations a clear learning effect might be perceivable if the democratic voting set up I designed

could result in a converging stationary result that makes sense. My expectation of a successful learning process would be that agents that do not belong the high skill minority (those agents who have a very hard time reaching income sufficient to be taxed by the second tax bracket) would vote for high taxes in the second tax bracket. Because these agents would have a majority and an interest in lower taxes for the first bracket, I would expect to see significantly higher taxes for the second tax bracket compared to the first bracket and a voting behavior where agents with lower skill vote for higher taxes in the second bracket compared to agents with higher skill.

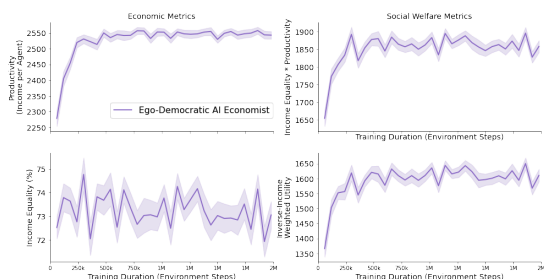


Figure 6: Training metrics of validation model (see also Figure 3)

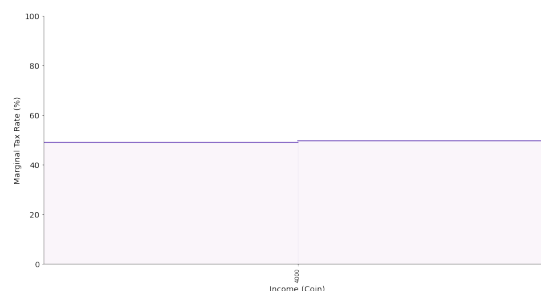


Figure 7: Average marginal tax rates of validation model (see also Figure 4)

The results after training do however not fulfill these expectations but instead show only marginally higher average tax rates for the higher tax brackets (see Figure 7) even though the training metrics (see Figure 6) seem to indicate that training reaches a plateau rather early. The impression that agents even in this simplified validation scenario do still pick their choices of marginal tax rates at random is further confirmed by investigating the actual marginal tax rates picked by agents of different skill levels after training.

The results in Figure 8 show that agents behavior closely resembles that of randomly picking marginal tax rates for each of the two brackets. Given the expectations above I would have expected the rightmost-upper graph to show a decreasing trend indicating that agents with lower skill are more likely to vote for higher tax rates in their own interest.

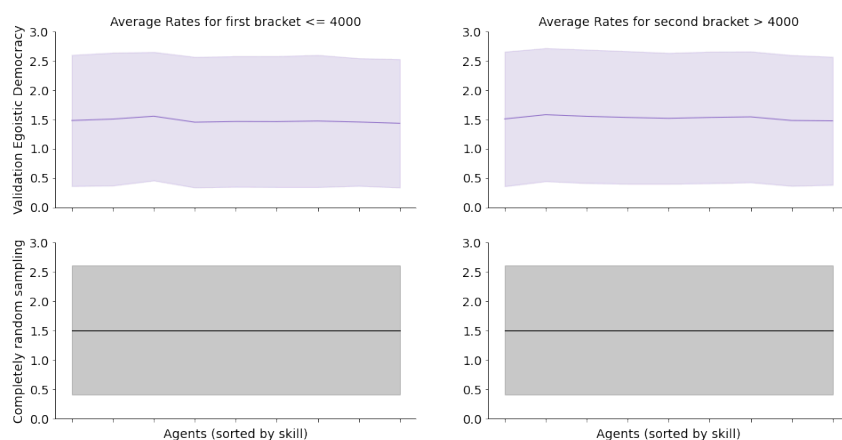


Figure 8: Average marginal tax rates picked with standard deviation indicated in the validation scenario

E. Discussion

While these results were rather disappointing for me personally, the stark difference in performance during training between the scenarios utilizing specialized roles for labor agents and an independent social planner compared to the new scenarios with a voting agent shows the superiority of agent specialization when it comes to MAS. The AI Economist Foundation framework only utilized a single policy that was updated for all mobile agents, which means all agents not the planner share the same learning process. Considering the problem of tax scheduling as a cooperative setting this means we operate in the field of homogenous team learning, where an concurrent approach with different policies for each agent might have resulted in better performance.

A major limitation of this work is the limited amount of training done, especially when compared to the results from the original paper (Zheng *et al.*, 2022) which inspired the project setup of this work. Further training might have led to more interesting and insightful results. The results from Section D.1 do however indicate that too little training might not be the only problem with regards to unexpected agent behavior.

Another limitation of this work is due to the setup of critical hyperparameters for training which were mostly adopted from (Zheng *et al.*, 2022). Other hyperparameters might have led to faster convergence during training and better results if properly configured. Due to limited personal knowledge in the application of MAS algorithms I was unable to manipulate most hyperparameters except for slight adjustments of the entropy schedule for PPO.

The newly created scenarios of democratic voting of tax brackets in this work can be considered partly cooperative and partly competitive. The scenario with voting by egoistic agents (see Section C.1.3) can be rather considered competitive while voting by (partly) utilitarian agents (see Section C.1.4) more closely resembles a cooperative setting. Discovering little to no difference between the behavior of agents in both settings if not completely explainable by insufficient training, might also illustrate the difficulty in differentiating between competitive and cooperative settings in complex economic environments as investigated here. It might be that in a labor market with egoistic agents deciding democratically about tax rates one can get a similarly beneficial tax schedule compared to the same scenario with more utilitarian minded agents. Due to the limited scope of this work and probably not enough training iterations, this requires further investigation to reach any earnest assessment.

F. Conclusion

In this work, after offering an overview into the topic of Multi-Agent Systems and the theoretical background surrounding Reinforcement Learning on this topic, I used the AI Economist Foundation framework (Zheng *et al.*, 2020) to construct scenarios of agents democratically voting for taxation schedules in a simplified economic environment and compared this to previous work on taxation scheduling using worker agents and a centralized planner (Zheng *et al.*, 2022). I found little difference between two scenarios utilizing voting agents for tax schedules even though they followed different reward functions that followed either a rational egoistic assumption about personal incentives and a (partly) utilitarian assumption of human motivation. After limited training both voting scenarios performed worse considering social welfare metrics and better when only considering productivity compared to setting a taxation schedule via a centralized planner or empirical calculations (using the Saez Formula (Saez, 2001)), but improved upon free market conditions. Further analysis showed that voting agents did not learn appropriate voting behavior of taxation schedules during the limited training process. Additional validation testing showed that this persisted even in a simplified scenario with additional training utilized. These results demonstrate among other factors the superiority of specialization of agents in MAS environments where training capabilities are limited.

Further research could extend the training duration and explore more sophisticated reinforcement learning algorithms to allow agents to better learn optimal voting behaviors and improve their ability to select beneficial tax schedules. For example (Zheng *et al.*, 2022) proposed a two-level RL approach where before taxation is introduced agents learn optimal labor behavior and taxation decisions are introduced with an annealing schedule. Additionally, the utilized scenario of a single-step-economy assumes relatively simple economic conditions. A natural extension would be to simulate more complex, dynamic economic environments like the Gather-Trade-Economy in (Zheng *et al.*, 2022) that can be utilized by the same framework. Additional extensions regarding taxation behavior might include investigating the effects of something like "lobbyism" where agents with higher income have more impact on decisions than agents with less income. It might also be of interest to examine the differences between directly deciding a tax schedule or choosing representatives that do so.

Bibliography

- Claus, C. and Boutilier, C. (1998) "The dynamics of reinforcement learning in cooperative multiagent systems," *AAAI/IAAI*, 1998(746–752), p. 2–3
- Fasli, M. (2007) *Agent technology for e-commerce*. Wiley
- Lauer, M. and Riedmiller, M. A. (2000) "An algorithm for distributed reinforcement learning in cooperative multi-agent systems," in *Proceedings of the seventeenth international conference on machine learning*, pp. 535–542
- Mnih, V. et al. (2013) "Playing Atari with Deep Reinforcement Learning," *CoRR* [Preprint]. Available at: <http://arxiv.org/abs/1312.5602>
- Panait, L. and Luke, S. (2005) "Cooperative multi-agent learning: The state of the art," *Autonomous agents and multi-agent systems*, 11, pp. 387–434
- Saez, E. (2001) "Using elasticities to derive optimal income tax rates," *The review of economic studies*, 68(1), pp. 205–229
- Schulman, J. et al. (2017) "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347* [Preprint]
- Wooldridge, M. (2009) *An introduction to multiagent systems*. John Wiley & sons
- Zheng, S. et al. (2020) "The AI Economist: Improving Equality and Productivity with AI-Driven Tax Policies"
- Zheng, S. et al. (2022) "The AI Economist: Taxation policy design via two-level deep multiagent reinforcement learning," *Science advances*, 8(18), p. eabk2607